

5.2.19

Taraka Abhinav - EE25BTECH11016

October 4, 2025

Question

Solve the system of equations:

$$x + y = 14 \quad (1)$$

$$x - y = 4 \quad (2)$$

Line Representation

The equation of a line is

$$\mathbf{n}^T \mathbf{x} = c \quad (3)$$

Line L1:

$$\begin{pmatrix} 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = 14 \quad (4)$$

Line L2:

$$\begin{pmatrix} 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = 4 \quad (5)$$

Matrix Form

These can be combined into the matrix form $A\mathbf{x} = \mathbf{b}$:

$$\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 14 \\ 4 \end{pmatrix} \quad (6)$$

The following augmented matrix can be solved by Gaussian elimination

$$\left(\begin{array}{cc|c} 1 & 1 & 14 \\ 1 & -1 & 4 \end{array} \right) \xrightarrow{R_2 \leftarrow R_2 - R_1} \left(\begin{array}{cc|c} 1 & 1 & 14 \\ 0 & -2 & -10 \end{array} \right) \quad (7)$$

Reduced Equations and Solution

The matrix is in row echelon form with two non-zero rows (Rank = 2). We can use back substitution. The second row gives:

$$-2y = -10 \implies y = 5 \quad (8)$$

Substituting $y = 5$ into the first row equation, $x + y = 14$:

$$x + 5 = 14 \implies x = 9 \quad (9)$$

The unique solution is:

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 9 \\ 5 \end{pmatrix} \quad (10)$$

Conclusion: The system has a **unique solution**.

C Code

```
#include <stdio.h>

typedef struct {
    double x, y;
} Point;

Point solve_linear_system() {
    // Solving manually for the given system
    Point p;
    //  $x + y = 14$ 
    //  $x - y = 4$ 
    p.x = (14 + 4)/2; // 9
    p.y = 14 - p.x; // 5
    return p;
}

int main() {
    Point P = solve_linear_system();
    printf("Intersection Point: P(%g, %g)\n", P.x, P.y);
```

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt

class Point(ctypes.Structure):
    _fields_ = [(x, ctypes.c_double), (y, ctypes.c_double)]

lib = ctypes.CDLL(./solve_linear.so)
lib.solve_linear_system.restype = Point

P = lib.solve_linear_system()

# Plotting
x_vals = np.linspace(0, 15, 300)
y_L = 14 - x_vals
y_K = x_vals - 4
```



```
plt.figure()
plt.plot(x_vals, y_L, label='L:  $x+y=14$ ')
plt.plot(x_vals, y_K, label='K:  $x-y=4$ ')

plt.scatter(P.x, P.y, color='red')
plt.text(P.x+0.3, P.y+0.3, f'P({int(P.x)},{int(P.y)})', color='red')

plt.xlim(0,15)
plt.ylim(0,15)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Solution of Linear System (C + Python)')
plt.grid(True)
plt.legend()
plt.show()
```

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Coefficients for lines
# Line L:  $x + y = 14$ 
# Line K:  $x - y = 4$ 

# Define x range for plotting
x_vals = np.linspace(0, 15, 300)

# Line L:  $y = 14 - x$ 
y_L = 14 - x_vals

# Line K:  $y = x - 4$ 
y_K = x_vals - 4
```

Python Code

```
# Intersection point (solving manually or using numpy)
A = np.array([[1, 1], [1, -1]])
b = np.array([14, 4])
sol = np.linalg.solve(A, b)
x_int, y_int = sol

# Plot lines
plt.figure()
plt.plot(x_vals, y_L, label='L: x+y=14')
plt.plot(x_vals, y_K, label='K: x-y=4')

# Mark intersection
plt.scatter(x_int, y_int, color='red')
plt.text(x_int+0.3, y_int+0.3, f'P({int(x_int)},{int(y_int)})',
        color='red')

plt.xlim(0,15)
plt.ylim(0,15)
plt.xlabel('x')
```

Plot

